

Encapsulamiento y abstracción

Introducción

La abstracción es una vista o representación de una entidad que incluye los atributos más significativos

En un sentido general permite agrupar entidades en grupos, teniendo en mente atributos esenciales.

En lenguajes de programación es una herramienta para reducir la complejidad de la programación (simplificando el proceso de programación)

Tipos fundamentales en la abstracción de lenguajes contemporáneos:
abstracción de procesos y **abstracción de datos**

Evolución del concepto de tipo: abstracción de datos

- Tipos básicos
- Tipos definidos por el programador
- Tipos de datos abstractos

Un **tipo de dato abstracto** es una entidad que encapsula valores y operaciones (solo acceso a través de las operaciones)

Un TDA genérico es una entidad que encapsula un conjunto de valores (debe incluir las operaciones) y un conjunto de valores de manera independiente.

Fortran:

Un tipo determina la interpretación de un valor almacenado y permite chequeos.

Pascal:

Un tipo es el conjunto de valor es que puede tomar un dato.

Simula:

Un tipo es un conjunto de valores y un conjunto de operaciones.

Abstracción

Abstracción de procesos

- Incorporado con unidades
- Mejora legibilidad
- Extensibilidad

Abstracción de datos

- Reduce la cantidad de detalles que el programador debe conocer y tener en mente a cada momento.
- Previene el uso incorrecto de componentes del programa (contención de fallas)
- Independencia significativa entre las componentes del programa.

Un **tipo de dato abstracto** es un tipo de dato definido por el usuario que satisface las siguientes condiciones:

1. La representación de los objetos del tipo son definidos como unidades sintácticas simples.
2. La representación del tipo son escondidos del programa que utilizan estos objetos, por lo tanto las únicas operaciones posibles son aquellas provistas por la definición del tipo

Abstracción datos

Un **tipo de dato abstracto** es un tipo de dato definido por el usuario que satisface las siguientes condiciones:

1. La representación de los objetos del tipo son definidos como unidades sintácticas simples.
2. La representación del tipo son escondidos del programa que utilizan estos objetos, por lo tanto las únicas operaciones posibles son aquellas provistas por la definición del tipo

La definición completa debería estar encapsulada de manera que el usuario del tipo solo necesite conocer el nombre del tipo, la semántica y las operaciones disponibles.

Abstracción de datos

El concepto central detrás del diseño de abstracciones definidas por el programados es el de **ocultamiento de información**.

Cuando la información es encapsulada en una abstracción, significa que el usuario de dicha abstracción:

- No necesita conocer la información oculta para poder hacer uso de la abstracción, y
- No está permitida la manipulación directa la información oculta aunque se desee hacerlo.

Conceptos importantes

El **ocultamiento de información** es una cuestión de diseño de programas. Es posible en cualquier programa que sea diseñado apropiadamente, independientemente del lenguaje de programación utilizado.

El **encapsulamiento** es una cuestión de diseño del lenguaje. Una abstracción resulta efectivamente encapsulada solo cuando el lenguaje prohíbe el acceso a la información oculta en la abstracción.

Consideraciones de diseño

- Unidad sintáctica para definir el tipo de dato abstracto.
- Operaciones built-in
 - Asignaciones
 - Comparación
- Operaciones comunes
 - Iteradores
 - Constructores
 - Destruyores
- Tipo de datos abstractos parametrizados
- Extensiones de tipos

Ejemplo en ADA

```
package Stack_Pack is
  type stack_type is limited private;
  max_size: constant := 100;
  function empty(stk: in stack_type) return Boolean;
  procedure push(stk: in out stack_type; elem:in Integer);
  procedure pop(stk: in out stack_type);
  function top(stk: in stack_type) return Integer;

  private -- hidden from clients
  type list_type is array (1..max_size) of Integer;
  type stack_type is record
    list: list_type;
    tosub: Integer range 0..max_size) := 0;
  end record;
end Stack_Pack
```

Ejemplo en C++

```
class stack {  
    private:  
        int *stackPtr, maxLen, topPtr;  
    public:  
        stack() { // a constructor  
            stackPtr = new int [100];  
            maxLen = 99;  
            topPtr = -1;  
        };  
        ~stack () {delete [] stackPtr;};  
        void push (int num) {...};  
        void pop () {...};  
        int top () {...};  
        int empty () {...};  
}
```

Parametrizado

Los tipos de datos parametrizados permiten el diseño de un tipo de dato abstracto que puede almacenar elementos de cualquier tipo.

Estos tipos de datos abstractos son también conocidos como **clases genéricas**.

Extensiones de tipos

Información que es conocida es una parte del programa generalmente es requerida y utilizada en otra parte. Eso sucede:

- **Explícitamente:** pasaje de parámetros, pasaje de información de los parámetros actuales a los formales. La ligadura se hace en la llamada explícita.
- **Implícitamente:** generalmente llamada **herencia**. Sucede cuando una componente del programa, recibe propiedades o características de otra componente siguiendo una relación especial, existente entre ambas componentes.

La abstracción relacionada con la herencia, puede verse como más que una pared que evita que el programador vea los contenidos de objetos de datos impropios.

La herencia provee el mecanismo para pasar información entre objetos de clases relacionadas. Hay cuatro relaciones que resumen el uso de la herencia en los lenguajes

¿Preguntas?
